

Day 1：开发基础与环境配置！

*今日时间安排

时段	时间	内容	形式
上午	9:00-10:30	Trae IDE安装与配置	动手实践
	10:45-12:00	Git版本控制基础	理论+实践
下午	14:00-15:30	Python环境搭建	动手实践
	15:45-17:00	Node.js生态系统	理论+实践
晚上	19:00-20:30	Docker Desktop入门	动手实践
	20:45-21:00	环境验收与总结	自测

✓✓ 学习目标

今日完成后，你将能够：

✓ 熟练使用Trae IDE - 掌握AI辅助编码、项目管理、调试功能 ✓ 掌握Git工作流 - 独立完成代码版本管理、分支操作、协作流程 ✓ 配置Python开发环境 - 使用pyenv/conda管理多版本Python ✓ 理解Node.js生态 - 掌握npm/pnpm包管理和NVM版本控制 ✓ 使用Docker容器化 - 能够编写Dockerfile、运行容器、管理镜像

** 详细学习内容

1. Trae IDE 安装与配置 (1.5小时)

1.1 为什么选择Trae IDE?

传统IDE的痛点：

- ！ 缺乏智能代码补全和上下文理解
- ！ 手动查找文档和API用法耗时
- ！ 调试效率低，问题定位困难

Trae IDE的优势:

- ☐ ✓ 内置AI助手, 实时提供代码建议
- ☐ ✓ 智能理解项目上下文, 精准补全
- ☐ ✓ 一键生成测试、重构代码、解释逻辑
- ☐ ✓ 原生支持Python、JavaScript、TypeScript等语言

1.2 安装步骤

```
# 1. 访问官网下载安装包
# https://www.trae.com/

# 2. 根据操作系统选择对应版本
# Windows: .exe 安装包
# macOS: .dmg 安装包
# Linux: .ApplImage 或 .deb/.rpm

# 3. 运行安装向导, 按提示完成安装

# 4. 首次启动配置
# - 选择主题 (推荐Dark Theme)
# - 配置编辑器字体 (推荐 Fira Code 或 JetBrains Mono)
# - 安装推荐插件 (Python、JavaScript、Docker支持)
```

1.3 核心功能实战

功能1: AI代码助手

```
# 示例: 使用Trae AI生成FastAPI应用骨架
# 输入注释或自然语言描述, AI自动生成代码

"""
创建一个FastAPI应用, 包含以下功能:
1. 用户注册接口 (POST /api/register)
2. 用户登录接口 (POST /api/login)
3. 使用SQLite数据库存储用户信息
4. 密码使用bcrypt加密
"""

# Trae AI会自动生成完整的代码实现
```

功能2: 智能重构

```
// 重构前：冗余的if-else判断
function getUserType(user) {
  if (user.role === 'admin') {
    return 'administrator'
  }
  * else if (user.role === 'editor') {
    return 'editor'
  }
  * else if (user.role === 'viewer') {
    return 'viewer'
  }
  * else {
    return 'guest'
  }
  *
}

// 使用Trae AI重构后：简洁的映射表
function getUserType(user) {
  const roleMap = {
    admin: 'administrator',
    editor: 'editor',
    viewer: 'viewer'
  }
  *
  return roleMap[user.role] || 'guest'
}
*
```

功能3：代码解释

```
# 选中复杂代码段，右键选择"Explain Code"
# AI会用通俗语言解释代码逻辑

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

# AI解释：
# 这是一个归并排序算法的实现。
# 1. 基线条件：如果数组长度*1，直接返回（已排序）
# 2. 分割阶段：将数组从中间分成左右两半
# 3. 递归调用：对左右子数组分别进行归并排序
# 4. 合并阶段：将两个有序数组合并成一个有序数组
# 时间复杂度：O(n log n)，空间复杂度：O(n)
```

1.4 常用快捷键

操作	Windows/Linux	macOS	说明
AI对话	Ctrl+Shift+I	Cmd+Shift+I	打开AI助手面板
代码补全	Tab	Tab	接受AI建议
快速修复	Ctrl+.	Cmd+.	显示修复建议
格式化	Shift+Alt+F	Shift+Option+F	格式化当前文件
终端	Ctrl+`	Ctrl+`	打开集成终端

2. Git 版本控制 (1.5小时)

2.1 Git核心概念

什么是Git?

- 分布式版本控制系统
- 记录代码的每一次变更
- 支持多人协作开发

三大区域:

工作区 (Working Directory)
* git add
暂存区 (Staging Area / Index)
* git commit
本地仓库 (Local Repository)
* git push
远程仓库 (Remote Repository)

2.2 基础命令实战

```
# ===== 初始化配置 =====

# 设置用户名和邮箱（首次使用必须配置）
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

# 查看配置
git config --list

# ===== 仓库基本操作 =====

# 初始化新仓库
cd your-project
git init

# 克隆远程仓库
git clone https://github.com/username/repo.git

# ===== 文件状态管理 =====

# 查看当前状态
git status

# 添加文件到暂存区
git add .                # 添加所有文件
git add filename.py      # 添加指定文件
git add *.py             # 添加所有Python文件

# 提交到本地仓库
git commit -m "feat: 添加用户认证功能"

# 查看提交历史
git log --oneline        # 简洁模式
git log --graph          # 图形化展示

# ===== 分支操作 =====

# 创建并切换分支
git checkout -b feature/login

# 或者使用新语法
git switch -c feature/login

# 查看所有分支
git branch -a            # 包含远程分支

# 合并分支
```

```
git checkout main
```

```
git merge feature/login
```


删除分支

```
git branch -d feature/login
```


===== 远程仓库操作 =====

查看远程仓库

```
git remote -v
```


添加远程仓库

```
git remote origin https://github.com/username/repo.git
```


推送到远程

```
git push origin main
```

```
git push -u origin feature/new-feature # 首次推送并跟踪
```


拉取更新

```
git pull origin main
```


===== 撤销操作 =====

撤销工作区修改（未add）

```
git checkout -- filename.py
```


撤销暂存区修改（已add未commit）

```
git reset HEAD filename.py
```


回退到上一个commit

```
git reset --hard HEAD*1    # ☐ 谨慎使用，会丢失代码
```

□ 2.3 .gitignore 配置

创建.gitignore 文件忽略不需要版本控制的文件：

```
# Python
__pycache__/*
*.py[cod]
*.so
.Python
env/
venv/

# Node.js
node_modules/
dist/
.env
.env.local

# IDE
.vscode/
.idea/
*.swp
*.swo

# OS
.DS_Store
Thumbs.db

# 项目特定
logs/
*.log
db.sqlite3
media/
```

□ 2.4 Git工作流最佳实践

Feature Branch工作流：

```
# 1. 从main创建功能分支
git checkout main
git pull origin main
git checkout -b feature/user-auth

# 2. 开发功能，频繁提交
git add .
git commit -m "feat: 实现用户注册接口"
git add .
git commit -m "feat: 添加邮箱验证功能"

# 3. 开发完成后，推送分支
git push -u origin feature/user-auth

# 4. 在GitHub/GitLab上创建Pull Request
# 5. 代码审查通过后，合并到main

# 6. 删除已合并的功能分支
git branch -d feature/user-auth
git push origin --delete feature/user-auth
```

Commit Message规范:

```
<type>(<scope>): <subject>

<body>

<footer>
```

类型说明: -feat: 新功能 -fix: Bug修复 -docs: 文档变更 -style: 代码格式调整 (不影响功能)
-refactor: 重构 -test: 测试相关 -chore: 构建/工具链相关

示例:

```
feat(auth): 添加JWT令牌刷新机制

- 实现access_token和refresh_token双令牌机制
- 添加令牌黑名单功能防止重放攻击
- 优化令牌过期时间配置

Closes #123
```

□3. Python 环境搭建 (1.5小时)

□3.1 为什么需要Python?

在AI Agent开发中，Python是绝对主流的语言：

✓ 丰富的AI生态 – PyTorch、TensorFlow、LangChain等 ✓ 简洁易读 – 伪代码般的语法，降低认知负担 ✓ 强大的数据处理 – NumPy、Pandas、Polars ✓ 活跃的社区 – 遇到问题容易找到解决方案

□3.2 版本管理工具选择

方案A: pyenv (推荐Linux/macOS)

```
# 安装pyenv (macOS使用Homebrew)
brew install pyenv

# 初始化shell配置
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.zshrc
echo '[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.zshrc
echo 'eval "$(pyenv init -)"' >> ~/.zshrc

# 安装Python版本
pyenv install 3.11.9
pyenv install 3.12.2

# 切换全局默认版本
pyenv global 3.11.9

# 为项目指定版本
cd your-project
pyenv local 3.12.2
```

方案B: conda (推荐Windows, 数据科学场景)

```
# 下载Miniconda安装器
# https://docs.conda.io/en/latest/miniconda.html

# 创建虚拟环境
conda create -n ai-agent python=3.11

# 激活环境
conda activate ai-agent

# 退出环境
conda deactivate

# 查看所有环境
conda env list

# 删除环境
conda env remove -n ai-agent
```

方案C: venv (Python内置, 轻量级)

```
# Python 3.3+ 内置, 无需额外安装

# 创建虚拟环境
python -m venv venv

# Windows激活
venv\Scripts\activate

# macOS/Linux激活
source venv/bin/activate

# 退出环境
deactivate
```

□ 3.3 项目依赖管理

requirements.txt 方式:

```
# 生成依赖文件
pip freeze > requirements.txt

# 安装依赖
pip install -r requirements.txt
```

poetry方式（现代化，推荐）：

```
# 安装poetry
pip install poetry

# 初始化项目
cd your-project
poetry init

# 添加依赖
poetry add fastapi uvicorn sqlalchemy
poetry add --group dev pytest black isort

# 安装所有依赖
poetry install

# 运行脚本
poetry run python main.py
```

pyproject.toml 示例：

```
[tool.poetry]
name = "ai-agent-project"
version = "0.1.0"
description = "AI Agent开发项目"
authors ["Your Name <your.email@example.com>"]

[tool.poetry.dependencies]
python = "^3.11"
fastapi = "^0.109.0"
uvicorn = {extras = ["standard"], version = "^0.27.0"}
sqlalchemy = "^2.0.25"
pydantic = "^2.6.0"
httpx = "^0.26.0"

[tool.poetry.group.dev.dependencies]
pytest = "^8.0.0"
black = "^24.1.1"
isort = "^5.13.2"
mypy = "^1.8.0"

[build-system]
requires = ["poetry-core"]
build-backend = "poetry.core.masonry.api"
```

□ 3.4 Python代码风格工具

```
# 安装代码格式化工具
pip install black isort flake8

# 格式化代码
black .
isort .

# 代码检查
flake8 .

# 在VSCode/Trae中配置保存时自动格式化
# settings.json:
{
  "editor.formatOnSave": true,
  "editor.defaultFormatter": "ms-python.black-formatter",
  "python.sortImports.args": ["--profile", "black"]
}
```

□4. Node.js 生态系统 (1.5小时)

□4.1 Node.js在AI Agent开发中的作用

虽然Python是AI主力语言，但Node.js在前端工具链和部分后端场景不可或缺：

✓ 前端构建 - Vite、Webpack等打包工具基于Node.js ✓ 全栈开发 - Next.js、Nuxt.js等服务端渲染框架 ✓ 工具脚本 - 项目自动化、代码生成等辅助工具 ✓ 实时应用 - WebSocket服务、实时通信等场景

□4.2 NVM 版本管理

```
# 安装nvm (Windows使用nvm-windows)
# https://github.com/coreybutler/nvm-windows/releases

# 安装Node.js版本
nvm install 18.19.0      # LTS版本
nvm install 20.11.0     # 当前LTS

# 切换版本
nvm use 18.19.0

# 设置默认版本
nvm alias default 18.19.0

# 查看已安装版本
nvm list
```

□4.3 包管理工具对比

npm (Node Package Manager):

```
# 初始化项目
npm init -y

# 安装依赖
npm install vue@3
npm install -D typescript # 开发依赖
npm install -g @vue/cli    # 全局安装

# 运行脚本
npm run dev
npm run build

# 更新依赖
npm update
npm outdated          # 检查过时的包
```

pnpm (性能更优, 节省磁盘空间):

```
# 安装pnpm
npm install -g pnpm

# 初始化项目
pnpm init

# 安装依赖
pnpm add vue@3
pnpm add -D vite          # 开发依赖

# 运行脚本
pnpm dev
pnpm build

# pnpm优势:
# 1. 速度快 (比npm快2-3倍)
# 2. 节省磁盘空间 (硬链接去重)
# 3. 严格的依赖管理 (避免幽灵依赖)
```

yarn (稳定可靠):

```
# 安装yarn
npm install -g yarn

# 使用方式与npm类似
yarn add react
yarn add -D webpack
yarn start
```

选择建议：

- ☐ 新项目推荐使用 pnpm（性能好、严格模式）
- ☐ 团队已有项目保持原有包管理器
- ☐ 个人项目可自由选择

4.4 package.json 详解

```
{
  "name": "ai-agent-frontend",
  "version": "0.1.0",
  "type": "module",                // ES Module模式
  "scripts": {
    "dev": "vite",                 // 开发服务器
    "build": "vite build",         // 生产构建
    "preview": "vite preview",     // 预览构建结果
    "lint": "eslint . --ext .vue,.js,.jsx,.cjs,.mjs --fix",
    "format": "prettier --write src/"
  },
  "dependencies": {                // 生产依赖
    "vue": "^3.4.15",
    "vue-router": "^4.2.5",
    "pinia": "^2.1.7",
    "axios": "^1.6.5"
  },
  "devDependencies": {             // 开发依赖
    "@vitejs/plugin-vue": "^5.0.3",
    "vite": "^5.0.11",
    "typescript": "^5.3.3",
    "eslint": "^8.56.0",
    "prettier": "^3.2.4"
  },
  "engines": {                    // 版本要求
    "node": ">=18.0.0",
    "pnpm": ">=8.0.0"
  }
}
```

□ 5. Docker Desktop 入门 (1.5小时)

□ 5.1 为什么需要Docker?

传统部署的痛点:

- ! "在我机器上能跑" - 环境不一致导致的问题
- ! 复杂的依赖安装 - 不同项目的环境冲突
- ! 部署困难 - 手动配置服务器环境繁琐

Docker的优势:

- ✓ 环境一致性 - 开发、测试、生产环境完全一致
- ✓ 快速部署 - 一条命令启动完整服务栈
- ✓ 资源隔离 - 容器间互不影响
- ✓ 易于扩展 - 水平扩容简单高效

□ 5.2 核心概念

Docker 镜像 (Image):

- +-- 只读模板 (类似ISO文件)
- +-- 包含运行应用所需的一切
- +-- 示例: python:3.11-slim, node:18-alpine

Docker 容器 (Container):

- +-- 镜像的运行实例 (类似虚拟机)
- +-- 可读可写层
- +-- 示例: 运行中的Web服务器

Dockerfile:

- +-- 构建镜像的指令脚本
- +-- 类似"菜谱"
- +-- 定义如何组装镜像

docker-compose.yml:

- +-- 多容器编排文件
- +-- 定义多个服务的关联关系
- +-- 一键启动整个应用栈

□ 5.3 Docker Desktop 安装

```
# 1. 下载Docker Desktop
# Windows: https://www.docker.com/products/docker-desktop/
# macOS: 同上（选择Apple Chip或Intel）

# 2. 安装并启动
# Windows可能需要启用WSL2 (Windows Subsystem for Linux)
# 安装过程中会自动配置

# 3. 验证安装
docker --version          # 查看Docker版本
docker-compose --version  # 查看Compose版本

# 4. 运行测试容器
docker run hello-world
```

□ 5.4 常用命令实战

===== 镜像操作 =====

搜索镜像
docker search python

拉取镜像
docker pull python:3.11-slim
docker pull node:18-alpine

查看本地镜像
docker images

删除镜像
docker rmi image-id

清理无用镜像
docker image prune

===== 容器操作 =====

运行容器（交互式）
docker run -it python:3.11-slim /bin/bash

运行容器（后台模式）
docker run -d --name my-app -p 8000:80 nginx

参数说明：
-d: 后台运行
--name: 容器名称
-p: 端口映射（主机端口:容器端口）

查看运行中的容器
docker ps

查看所有容器（包括停止的）
docker ps -a

停止容器
docker stop container-name

启动已停止的容器
docker start container-name

删除容器
docker rm container-name

进入运行中的容器
docker exec -it container-name /bin/bash

查看容器日志

```
docker logs -f container-name # -f 实时跟踪日志
```


===== 数据卷操作 =====

创建数据卷

```
docker volume create my-data
```


挂载数据卷（数据持久化）

```
docker run -v my-data:/app/data python:3.11-slim
```


挂载本地目录

```
docker run -v $(pwd):/app python:3.11-slim
```


===== 网络操作 =====

查看网络列表

docker network ls

创建网络

```
docker network create my-network
```


让容器加入网络

```
docker run --network my-network app1
```

```
docker run --network my-network app2
```

□ 5.5 编写Dockerfile

Python应用示例:

```
# 基础镜像
FROM python:3.11-slim

# 设置工作目录
WORKDIR /app

# 设置环境变量
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

# 复制依赖文件
COPY requirements.txt .

# 安装依赖
RUN pip install --no-cache-dir -r requirements.txt

# 复制应用代码
COPY . .

# 暴露端口
EXPOSE 8000

# 启动命令
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Node.js应用示例:

```
# 构建阶段
FROM node:18-alpine AS builder

WORKDIR /app

# 先复制package文件，利用缓存
COPY package.json pnpm-lock.yaml ./
RUN npm install -g pnpm && pnpm install --frozen-lockfile

# 复制源码并构建
COPY . .
RUN pnpm build

# 生产阶段
FROM node:18-alpine AS production

WORKDIR /app

COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./

EXPOSE 3000

CMD ["node", "dist/server.js"]
```

优化技巧:

```
# 1. 使用更小的基础镜像
FROM python:3.11-slim          # 好 (291MB)
# vs
FROM python:3.11              # 差 (1GB+)

# 2. 利用构建缓存 (先复制依赖文件)
COPY requirements.txt .        # 变化少, 能命中缓存
RUN pip install -r requirements.txt
COPY . .                      # 变化多, 放在后面

# 3. 多阶段构建 (减小最终镜像体积)
# 见上面的Node.js示例

# 4. 减少层数 (合并RUN指令)
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        gcc && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*
```

□ 5.6 Docker Compose编排

docker-compose.yml 示例:

```
version: '3.8'
```

```
services:
```

```
  # 后端API服务
```

```
  backend:
```

```
    build: ./backend
```

```
    ports:
```

```
      - "8000:8000"
```

```
    environment:
```

```
      - DATABASE_URL=postgresql://user:pass@db:5432/mydb
```

```
      - REDIS_URL=redis://redis:6379
```

```
    depends_on:
```

```
      - db
```

```
      - redis
```

```
    volumes:
```

```
      - ./backend:/app
```

```
    restart: unless-stopped
```

```
  # 前端服务
```

```
  frontend:
```

```
    build: ./frontend
```

```
    ports:
```

```
      - "3000:80"
```

```
    depends_on:
```

```
      - backend
```

```
    restart: unless-stopped
```

```
  # PostgreSQL数据库
```

```
  db:
```

```
    image: postgres:15-alpine
```

```
    environment:
```

```
      POSTGRES_USER: user
```

```
      POSTGRES_PASSWORD: pass
```

```
      POSTGRES_DB: mydb
```

```
    volumes:
```

```
      - postgres_data:/var/lib/postgresql/data
```

```
    ports:
```

```
      - "5432:5432"
```

```
    restart: unless-stopped
```

```
  # Redis缓存
```

```
  redis:
```

```
    image: redis:7-alpine
```

```
    ports:
```

```
      - "6379:6379"
```

```
    volumes:
```

```
      - redis_data:/data
```

```
    restart: unless-stopped
```

volumes:

postgres_data:

redis_data:

常用Compose命令：

```
# 启动所有服务（后台运行）
docker-compose up -d

# 查看服务状态
docker-compose ps

# 查看日志
docker-compose logs -f backend

# 重启某个服务
docker-compose restart backend

# 停止所有服务
docker-compose down

# 停止并删除数据卷
docker-compose down -v

# 重新构建并启动
docker-compose up -d --build
```

*> 实践任务清单

□任务1：初始化项目仓库（30分钟）

```
# 1. 在GitHub创建新仓库 ai-agent-week
# 2. 克隆到本地
git clone https://github.com/username/ai-agent-week.git
cd ai-agent-week

# 3. 创建初始README.md
# 4. 配置.gitignore
# 5. 创建Day1文件夹结构
mkdir -p day01/environment-setup
cd day01/environment-setup

# 6. 创建第一个提交
git add .
git commit -m "chore: 初始化项目仓库"
git push origin main
```

检验标准：

- ☐ [] GitHub仓库可见且包含README.md
- ☐ [] 本地能正常push/pull
- ☐ [] .gitignore配置正确（忽略venv、node_modules等）

☐任务2：配置Python开发环境（45分钟）

```
# 1. 使用conda/pyenv创建虚拟环境
conda create -n ai-agent python=3.11
conda activate ai-agent

# 2. 初始化项目依赖
pip install fastapi uvicorn sqlalchemy httpx pytest black

# 3. 生成requirements.txt
pip freeze > requirements.txt

# 4. 创建简单的FastAPI应用
# main.py
from fastapi import FastAPI

app = FastAPI(title="AI Agent Starter")

@app.get("/")
async def root():
    return {"message": "Hello AI Agent!"}

@app.get("/health")
async def health_check():
    return {"status": "healthy"}

# 5. 运行测试
uvicorn main:app --reload
```

检验标准：

- ☐ [] 虚拟环境能正常激活和退出
- ☐ [] 能正常安装和导入依赖包
- ☐ [] FastAPI应用能在 <http://localhost:8000> 访问
- ☐ [] 访问 <http://localhost:8000/docs> 能看到Swagger文档

☐任务3：配置前端开发环境（45分钟）

```
# 1. 使用pnpm创建Vue3项目
pnpm create vite@latest frontend -- --template vue-ts

# 2. 进入项目并安装依赖
cd frontend
pnpm install

# 3. 安装常用依赖
pnpm add vue-router pinia axios element-plus

# 4. 启动开发服务器
pnpm dev

# 5. 验证页面能正常访问（通常是 http://localhost:5173）
```

检验标准：

- ☐ [] Vue3项目能正常运行
- ☐ [] 开发服务器热更新正常工作
- ☐ [] TypeScript类型检查无报错

☐任务4: Docker化应用（60分钟）

```
# 1. 为后端创建Dockerfile
# （参考上文5.5节）

# 2. 为前端创建Dockerfile
# （参考上文5.5节）

# 3. 创建docker-compose.yml
# （参考上文5.6节）

# 4. 构建并启动
docker-compose up -d --build

# 5. 验证服务
# - 访问 http://localhost:8000/docs（后端API）
# - 访问 http://localhost:3000（前端页面）

# 6. 提交Docker相关文件
git add Dockerfile docker-compose.yml .dockerignore
git commit -m "feat: 添加Docker容器化配置"
```

检验标准：

- ☐ [] `docker-compose up -d` 能一键启动全部服务
- ☐ [] 所有服务都能正常访问
- ☐ [] `docker-compose ps` 显示所有服务状态为Up
- ☐ [] 日志无严重错误输出

** 推荐学习资源

☐ 官方文档

- ☐ Trae IDE 官方文档 - 全面了解IDE功能
- ☐ Pro Git 中文版 - Git权威指南
- ☐ Python 官方教程 - Python入门必读
- ☐ Node.js 官方文档 - Node.js API参考
- ☐ Docker 实战文档 - Docker入门到进阶

☐ 视频教程

- ☐ Git: 尚硅谷Git教程 (B站) - 系统全面
- ☐ Python: Python编程从入门到实践 (慕课网) - 实战导向
- ☐ Docker: Docker快速上手 (YouTube/Docker官方频道)

☐ GitHub项目

- ☐ git-tips - Git实用技巧集合
- ☐ awesome-docker - Docker资源大全
- ☐ python-guide - Python最佳实践

☐ 练习平台

- ☐ Git Immersion - Git交互式学习
- ☐ Katacoda Docker Playground - Docker在线实验
- ☐ Python Tutor - Python代码可视化执行

✓ 今日自测题

☐ 选择题 (每题10分, 共100分)

- ☐ Git中, `git add` 的作用是什么?
- ☐ A. 提交代码到远程仓库
- ☐ B. 将文件添加到暂存区
- ☐ C. 创建新的分支
- ☐ D. 查看代码差异
- ☐ 以下哪个不是Docker的核心概念?

- ☐ A. Image（镜像）
- ☐ B. Container（容器）
- ☐ C. Volume（卷）
- ☐ D. Namespace（命名空间）!! 这是Linux概念

☐ pnpm相比npm的主要优势是什么？

- ☐ A. 功能更多
- ☐ B. 速度更快且节省磁盘空间
- ☐ C. 社区更大
- ☐ D. 支持更多语言

☐ Python虚拟环境的作用是什么？

- ☐ A. 加速代码运行
- ☐ B. 隔离项目依赖，避免冲突
- ☐ C. 自动安装所有库
- ☐ D. 代码加密保护

☐ Dockerfile中COPY 和ADD 指令的区别？

- ☐ A. 没有区别
- ☐ B. ADD支持URL和自动解压tar包
- ☐ C. COPY速度更快
- ☐ D. COPY支持通配符

☐ docker-compose up -d 中-d 参数的含义？

- ☐ A. 调试模式
- ☐ B. 后台运行
- ☐ C. 删除容器
- ☐ D. 详细日志

☐ NVM的主要用途是什么？

- ☐ A. 管理npm包
- ☐ B. 管理Node.js版本
- ☐ C. 运行Node.js脚本
- ☐ D. 打包Node.js应用

☐ .gitignore 文件的作用？

- ☐ A. 忽略代码错误
- ☐ B. 指定不被Git追踪的文件
- ☐ C. 自动生成文档
- ☐ D. 配置Git用户信息

☐ 以下哪个是Python推荐的依赖管理工具？

- ☐ A. pip + requirements.txt
 - ☐ B. poetry
 - ☐ C. pipenv
 - ☐ D. 以上都是（不同场景适用）
-
- ☐ Docker容器的特点不包括？
 - ☐ A. 轻量级
 - ☐ B. 环境一致性
 - ☐ C. 强隔离性（类似虚拟机）!! 容器隔离性不如VM
 - ☐ D. 快速启动

☐答案与解析

- ☐ 答案: B -git add 将工作区的改动添加到暂存区, 准备提交。
- ☐ 答案: D - Namespace是Linux内核概念, 不是Docker特有概念（虽然Docker使用了它）。
- ☐ 答案: B - pnpm使用硬链接和符号链接, 速度快且节省磁盘空间。
- ☐ 答案: B - 虚拟环境为每个项目创建独立的Python解释器和包空间。
- ☐ 答案: B - ADD额外支持从URL下载文件和自动解压压缩包。
- ☐ 答案: B --d 表示detached mode（分离模式）, 即后台运行。
- ☐ 答案: B - NVM（Node Version Manager）用于安装和切换不同版本的Node.js。
- ☐ 答案: B -.gitignore 指定Git应该忽略的文件和目录模式。
- ☐ 答案: D - 三种工具各有优劣, 可根据团队习惯和项目需求选择。
- ☐ 答案: C - 容器的隔离性是基于进程级别的, 不如虚拟机强隔离。

评分标准:

- ☐ 90-100分: √>> 优秀! 可以进入下一阶段
- ☐ 70-89分: *☐良好! 建议复习错题知识点
- ☐ 60-69分: !! 及格! 需要加强练习
- ☐ 60分以下: *☐建议重新学习今日内容

** 今日总结

☐关键收获

请在下方记录今天学到的最重要的3个知识点:

- ☐ _____
- ☐ _____
- ☐ _____

☐遇到的挑战

记录今天遇到的技术难点及解决方案:

- ☐ 问题：解决：
- ☐ 问题：解决：

☐明日预告

Day 2: 前端技术复习与强化 将涵盖：

- ☐ HTML5语义化标签的最佳实践
- ☐ CSS现代布局技术（Flexbox/Grid）
- ☐ JavaScript ES6+核心特性
- ☐ Vue3 Composition API深度学习
- ☐ 组件化开发与状态管理

准备工作：

- ☐ [] 确保Vue3开发环境就绪
- ☐ [] 预习HTML5和CSS基础知识
- ☐ [] 准备好一个待实现的Vue3小项目思路

*☐扩展资源

☐进阶学习

- ☐ Git高级命令 - rebase、cherry-pick等
- ☐ Docker安全最佳实践 - 生产环境注意事项
- ☐ Python性能优化 - 代码性能提升技巧

☐工具推荐

- ☐ Git GUI工具：SourceTree、GitKraken（可视化Git操作）
- ☐ 终端增强：Oh My Zsh、Windows Terminal（提升命令行体验）
- ☐ API测试：Postman、Insomnia（测试REST API）
- ☐ 容器管理：Portainer（Docker可视化界面）

√* Day 1 完成！你已具备AI Agent开发的坚实基础！

明日将继续探索现代前端技术，为全栈能力添砖加瓦！